

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Алгоритмы ориентации в пространстве

Отчёт по лабораторной работе № 4

Работу
выполнил:
Сухов Н. М.
Группа: R3335
Преподаватель:
Каплин А. В.

Санкт-Петербург
2026

Содержание

1. Цель работы	3
2. Краткая теория	3
2.1. Вектор состояния EKF-SLAM	3
2.2. Модель движения робота	3
2.3. Модель измерений	4
2.4. Формулы EKF predict/update	5
2.5. Явные выражения якобианов	5
3. Описание реализации	6
3.1. Структура программы	6
3.2. Инициализация неизвестных меток	6
3.3. Используемые параметры	7
3.4. Особенности и допущения	7
4. Результаты моделирования	8
4.1. Показатели качества	8
4.2. Базовый эксперимент	8
4.3. Таблица параметров экспериментов	12
4.4. Исследование чувствительности	12
5. Выводы	13
6. Исходный код	14

1. Цель работы

Целью лабораторной работы является изучение алгоритма одновременной локализации и картирования на основе расширенного фильтра Калмана. В ходе работы необходимо реализовать EKF-SLAM для модели робота с управлением линейной и угловой скоростями и датчиком дальности/азимута до статических меток.

В работе требуется оценить точность, ковариацию и сходимость оценки положения робота и координат меток, а также проанализировать результаты при разных уровнях шума и различном радиусе наблюдения.

Выполняется полный вариант EKF-SLAM, при котором координаты меток заранее неизвестны. При первом наблюдении метка инициализируется по измерению дальности и азимута, после чего ее координаты добавляются в общий вектор состояния фильтра.

2. Краткая теория

2.1. Вектор состояния EKF-SLAM

В плоской задаче состояние робота задается вектором

$$\mathbf{x}_r = [x \ y \ \theta]^T$$

где x и y - координаты робота в мировой системе координат, θ - угол ориентации робота.

Координаты i -й метки задаются вектором

$$\mathbf{m}_i = [x_i \ y_i]^T$$

Полный вектор состояния EKF-SLAM включает положение робота и координаты всех инициализированных меток:

$$\mathbf{X} = [x \ y \ \theta \ x_1 \ y_1 \ x_2 \ y_2 \ \dots \ x_N \ y_N]^T$$

Размерность полного состояния равна

$$n_X = 3 + 2N$$

где N - число меток, уже добавленных в карту.

2.2. Модель движения робота

Управляющее воздействие задается вектором

$$\mathbf{u}_k = [v_k \ \omega_k]^T$$

где v_k - линейная скорость робота, ω_k - угловая скорость робота.

Дискретная модель движения за интервал времени Δt имеет вид

$$x_{k+1} = x_k + v_k \Delta t \cos \theta_k + w_x$$

$$y_{k+1} = y_k + v_k \Delta t \sin \theta_k + w_y$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t + w_\theta$$

где w_x , w_y , w_θ - компоненты шума процесса.

Шум процесса считается нормально распределенным:

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{R})$$

Без учета шума функция перехода состояния для части, относящейся к роботу, записывается следующим образом:

$$f(\mathbf{x}_{r,k}, \mathbf{u}_k) = \begin{bmatrix} x_k + v_k \Delta t \cos \theta_k \\ y_k + v_k \Delta t \sin \theta_k \\ \theta_k + \omega_k \Delta t \end{bmatrix}$$

Так как метки считаются неподвижными, на этапе прогноза изменяется только состояние робота, а координаты уже добавленных меток остаются неизменными.

Якобиан модели движения по состоянию робота равен

$$\mathbf{F}_r = \begin{bmatrix} 1 & 0 & -v_k \Delta t \sin \theta_k \\ 0 & 1 & v_k \Delta t \cos \theta_k \\ 0 & 0 & 1 \end{bmatrix}$$

Для полного вектора состояния матрица $\mathbf{F}_k$ имеет блочно-диагональный вид:

$$\mathbf{F}_k = \begin{bmatrix} \mathbf{F}_r & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{2N} \end{bmatrix}$$

2.3. Модель измерений

Датчик робота измеряет дальность и азимут до наблюдаемой метки:

$$\mathbf{z}_j = [r_j \quad \varphi_j]^T$$

Для j -й метки введем обозначения

$$d_x = x_j - x, \quad d_y = y_j - y, \quad q = d_x^2 + d_y^2$$

Тогда модель измерения дальности и азимута имеет вид

$$r_j = \sqrt{d_x^2 + d_y^2} + n_r$$

$$\varphi_j = \text{atan2}(d_y, d_x) - \theta + n_\varphi$$

где n_r и n_φ - шумы измерений.

Шум измерений считается нормально распределенным:

$$\mathbf{n}_k \sim \mathcal{N}(0, \mathbf{Q})$$

Нелинейная функция измерений без учета шума записывается как

$$h_j(\mathbf{X}) = \begin{bmatrix} \sqrt{(x_j - x)^2 + (y_j - y)^2} \\ \text{atan2}(y_j - y, x_j - x) - \theta \end{bmatrix}$$

2.4. Формулы EKF predict/update

На этапе прогноза рассчитывается априорная оценка состояния:

$$\hat{\mathbf{X}}_{k|k-1} = f\left(\hat{\mathbf{X}}_{k-1|k-1}, \mathbf{u}_k\right)$$

Ковариационная матрица ошибки прогноза определяется выражением

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \bar{\mathbf{R}}_k$$

где $\bar{\mathbf{R}}_k$ - матрица шума процесса, расширенная до размерности полного вектора состояния.

На этапе коррекции для каждой наблюдаемой метки рассчитывается невязка измерений:

$$\mathbf{y}_k = \mathbf{z}_k - h\left(\hat{\mathbf{X}}_{k|k-1}\right)$$

Так как второй компонентой измерения является угол, невязка азимута нормализуется к диапазону $(-\pi, \pi]$.

Ковариация невязки равна

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{Q}$$

Калмановский коэффициент определяется выражением

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1}$$

Коррекция оценки состояния выполняется по формуле

$$\hat{\mathbf{X}}_{k|k} = \hat{\mathbf{X}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k$$

Для обновления ковариационной матрицы используется форма Джозефа:

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{Q} \mathbf{K}_k^T$$

2.5. Явные выражения якобианов

Для j -й метки блок якобиана модели измерений по состоянию робота имеет вид

$$\mathbf{H}_{r,j} = \begin{bmatrix} -\frac{d_x}{\sqrt{q}} & -\frac{d_y}{\sqrt{q}} & 0 \\ \frac{d_y}{q} & -\frac{d_x}{q} & -1 \end{bmatrix}$$

Блок якобиана по координатам метки имеет вид

$$\mathbf{H}_{m,j} = \begin{bmatrix} \frac{d_x}{\sqrt{q}} & \frac{d_y}{\sqrt{q}} \\ -\frac{d_y}{q} & \frac{d_x}{q} \end{bmatrix}$$

Полная матрица $\mathbf{H}_k$ имеет размер $2 \times (3 + 2N)$. Все ее элементы равны нулю, кроме столбцов, соответствующих состоянию робота и координатам наблюдаемой метки:

$$\mathbf{H}_k = [\mathbf{H}_{r,j} \quad \mathbf{0} \quad \dots \quad \mathbf{H}_{m,j} \quad \dots \quad \mathbf{0}]$$

Правильная индексация блоков $\mathbf{H}_{r,j}$ и $\mathbf{H}_{m,j}$ является одним из ключевых условий корректной работы EKF-SLAM.

3. Описание реализации

3.1. Структура программы

В программе реализованы следующие основные этапы:

1. задание двумерной среды и координат статических меток;
2. моделирование истинного движения робота по заданным управлениям;
3. генерация зашумленных измерений дальности и азимута до видимых меток;
4. прогноз состояния и ковариационной матрицы по модели движения;
5. инициализация новых меток при первом наблюдении;
6. коррекция состояния и ковариационной матрицы по каждому измерению;
7. расчет ошибок и показателей качества;
8. построение и сохранение графиков в папку `img`.

3.2. Инициализация неизвестных меток

В полном EKF-SLAM координаты меток заранее неизвестны. Если измерение до новой метки равно $\mathbf{z} = [r, \varphi]^T$, то ее глобальные координаты рассчитываются по формулам

$$m_x = x + r \cos(\theta + \varphi)$$

$$m_y = y + r \sin(\theta + \varphi)$$

После этого координаты m_x и m_y добавляются в конец вектора состояния $\mathbf{X}$, а матрица $\mathbf{P}$ расширяется на две строки и два столбца.

Для вычисления ковариации новой метки используются якобианы преобразования наблюдения в глобальные координаты:

$$\mathbf{G}_x = \begin{bmatrix} 1 & 0 & -r \sin(\theta + \varphi) \\ 0 & 1 & r \cos(\theta + \varphi) \end{bmatrix}$$

$$\mathbf{G}_z = \begin{bmatrix} \cos(\theta + \varphi) & -r \sin(\theta + \varphi) \\ \sin(\theta + \varphi) & r \cos(\theta + \varphi) \end{bmatrix}$$

Блок ковариации новой метки определяется выражением

$$\mathbf{P}_{mm} = \mathbf{G}_x \mathbf{P}_{rr} \mathbf{G}_x^T + \mathbf{G}_z \mathbf{Q} \mathbf{G}_z^T$$

Перекрестная ковариация новой метки с уже существующим состоянием задается формулой

$$\mathbf{P}_{mX} = \mathbf{G}_x \mathbf{P}_{rX}$$

3.3. Используемые параметры

В среде было задано восемь статических меток:

$$N_m = 8$$

Количество шагов моделирования:

$$N = 700$$

Шаг дискретизации:

$$\Delta t = 0.1 \text{ с}$$

Общее время моделирования:

$$T = N\Delta t = 70 \text{ с}$$

Базовый радиус видимости меток:

$$r_{vis} = 6.5 \text{ м}$$

Начальное истинное состояние робота:

$$\mathbf{x}_{true,0} = [0 \quad 0 \quad 8^\circ]^T$$

Начальная оценка состояния робота:

$$\hat{\mathbf{x}}_0 = [0.25 \quad -0.25 \quad 4^\circ]^T$$

Начальная ковариационная матрица робота:

$$\mathbf{P}_0 = \text{diag} (0.35^2, 0.35^2, (8^\circ)^2)$$

Базовая ковариационная матрица шума процесса:

$$\mathbf{R} = \text{diag} (0.02^2, 0.02^2, (0.7^\circ)^2)$$

Базовая ковариационная матрица шума измерений:

$$\mathbf{Q} = \text{diag} (0.12^2, (1.5^\circ)^2)$$

Управляющее воздействие задавалось функциями

$$v(t) = 0.75 + 0.08 \sin(0.20t)$$

$$\omega(t) = 0.19 + 0.04 \sin(0.17t)$$

3.4. Особенности и допущения

В реализации приняты следующие допущения:

1. метки являются неподвижными;
2. идентификатор наблюдаемой метки считается известным, поэтому задача сопоставления наблюдений с метками отдельно не решается;
3. измерения формируются только для меток, находящихся внутри заданного радиуса видимости;
4. углы после прогноза и коррекции нормализуются к диапазону $(-\pi, \pi]$;
5. ковариационная матрица обновляется в форме Джозефа для повышения численной устойчивости.

4. Результаты моделирования

4.1. Показатели качества

Для оценки точности определения положения робота использовалась среднеквадратическая ошибка положения:

$$RMSE_{pos} = \sqrt{\frac{1}{N} \sum_{k=1}^N [(x_k - \hat{x}_k)^2 + (y_k - \hat{y}_k)^2]}$$

Также отдельно рассчитывались ошибки по координатам x , y и углу ориентации θ :

$$RMSE_x = \sqrt{\frac{1}{N} \sum_{k=1}^N (x_k - \hat{x}_k)^2}$$

$$RMSE_y = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_k - \hat{y}_k)^2}$$

$$RMSE_\theta = \sqrt{\frac{1}{N} \sum_{k=1}^N (\theta_k - \hat{\theta}_k)^2}$$

Для оценки качества построения карты использовалась средняя ошибка координат меток:

$$\bar{e}_m = \frac{1}{N_m} \sum_{j=1}^{N_m} \sqrt{(x_j - \hat{x}_j)^2 + (y_j - \hat{y}_j)^2}$$

Дополнительно рассчитывался критерий NIS:

$$NIS_k = \mathbf{y}_k^T \mathbf{S}_k^{-1} \mathbf{y}_k$$

Критерий NIS показывает, насколько фактическое измерение отличается от ожидаемого измерения с учетом ковариации невязки.

4.2. Базовый эксперимент

На рис. 4.1 представлены истинная и оцененная траектории робота, а также истинные и оцененные положения меток. Для оцененных меток построены ковариационные эллипсы. Видно, что после инициализации меток фильтр постепенно уточняет как положение робота, так и карту окружающей среды.

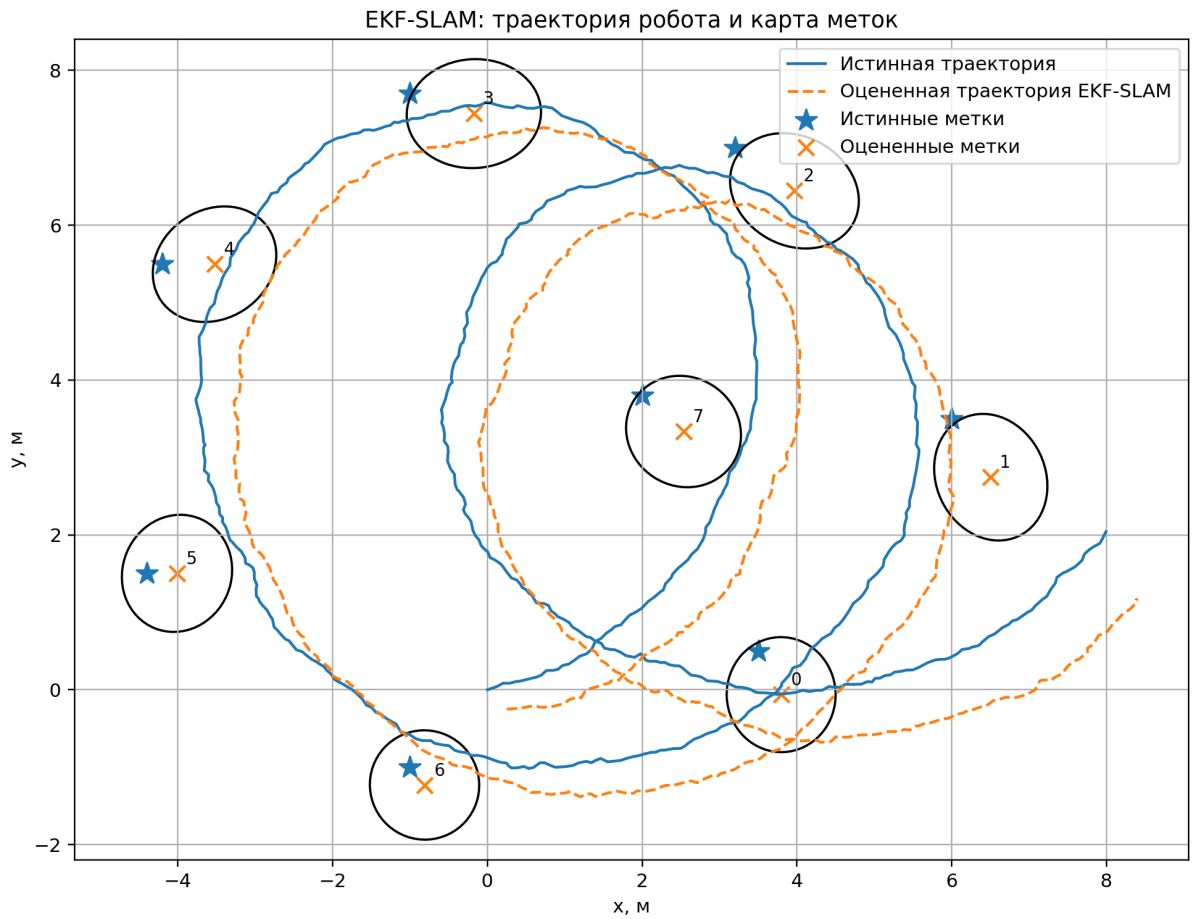


Рисунок 4.1. Истинная и оцененная траектории робота, истинные и оцененные метки

На рис. 4.2 представлены ошибки оценки координат x , y и угла ориентации θ во времени. В начале моделирования ошибки связаны с несовпадением начального истинного состояния и начальной оценки. Далее после накопления измерений ошибки ограничиваются и не имеют неустойчивого роста.



Рисунок 4.2. Ошибки оценки состояния робота во времени

На рис. 4.3 показано изменение накопленного RMSE положения робота. После начального переходного участка значение ошибки стабилизируется, что указывает на сходимость оценки.

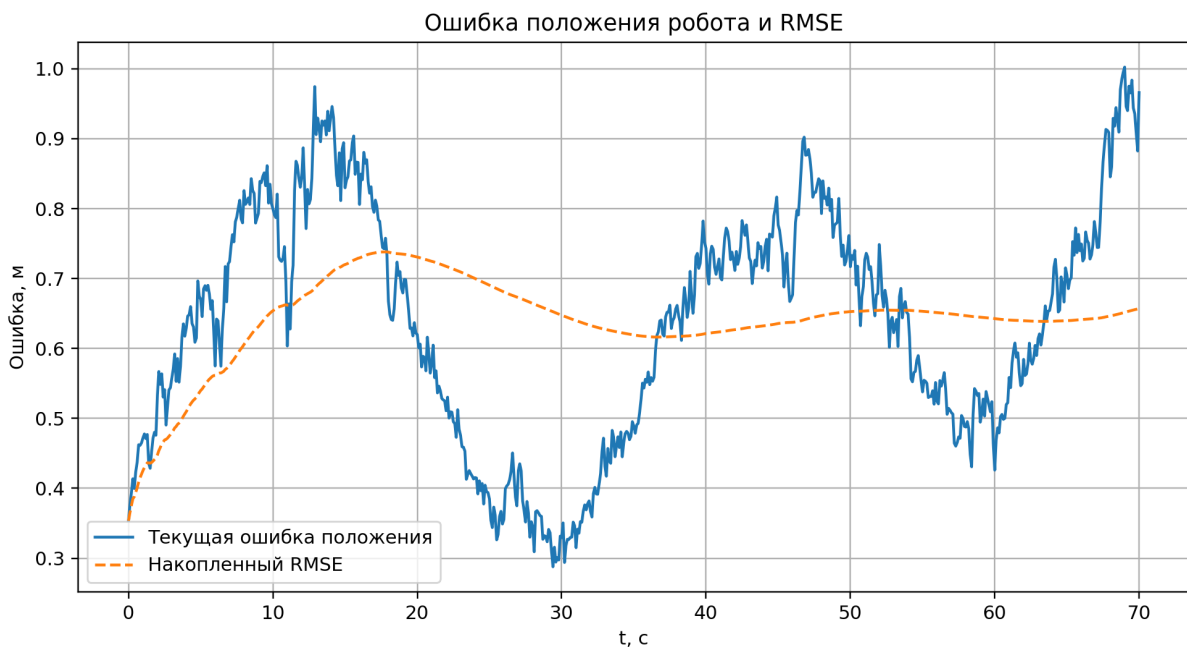


Рисунок 4.3. Накопленное значение RMSE положения робота

На рис. 4.4 представлено количество видимых меток на каждом шаге моделирования. Число наблюдаемых меток изменяется по мере движения робота, что влияет на интенсивность коррекции фильтра.



Рисунок 4.4. Количество видимых меток во времени

На рис. 4.5 показаны значения критерия NIS. Значения NIS в основном находятся в диапазоне нескольких единиц, что соответствует ожидаемому порядку величины для измерений дальности и азимута.

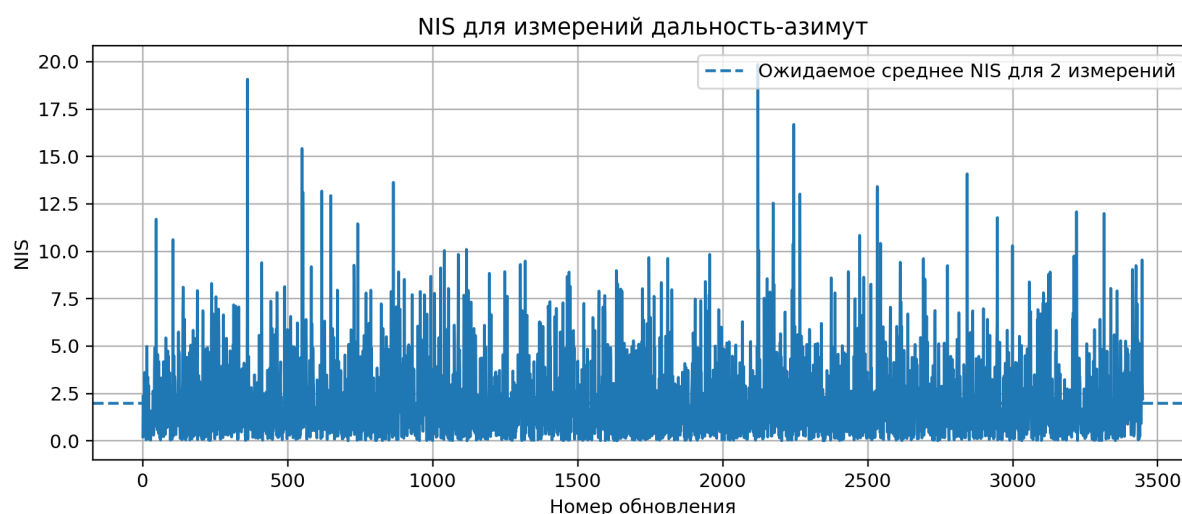


Рисунок 4.5. Значения критерия NIS во времени

Для базового эксперимента были получены следующие итоговые показатели:

$$RMSE_{pos} = 0.6569 \text{ м}$$

$$RMSE_x = 0.4808 \text{ м}$$

$$RMSE_y = 0.4477 \text{ м}$$

$$RMSE_{\theta} = 3.6529^{\circ}$$

$$\bar{e}_m = 0.6777 \text{ м}$$

$$\overline{NIS} = 2.0445$$

Всего в процессе моделирования были инициализированы все восемь меток. Среднее число видимых меток составило

$$\bar{N}_{vis} = 4.939$$

4.3. Таблица параметров экспериментов

Для анализа чувствительности алгоритма были выполнены четыре эксперимента:

1. базовый эксперимент;
2. эксперимент с увеличенной в 4 раза ковариационной матрицей шума измерений $\mathbf{Q}$;
3. эксперимент с увеличенной в 4 раза ковариационной матрицей шума процесса $\mathbf{R}$;
4. эксперимент с уменьшенным радиусом видимости до 4 м.

Сравниваемые параметры экспериментов приведены в табл. 4.1.

Таблица 4.1

Параметры экспериментов EKF-SLAM

Эксперимент	Шум процесса	Шум измерений	Радиус видимости, м
Базовый	$\mathbf{R}$	$\mathbf{Q}$	6.5
Увеличенный $\mathbf{Q}$	$\mathbf{R}$	$4\mathbf{Q}$	6.5
Увеличенный $\mathbf{R}$	$4\mathbf{R}$	$\mathbf{Q}$	6.5
Радиус 4 м	$\mathbf{R}$	$\mathbf{Q}$	4.0

4.4. Исследование чувствительности

На рис. 4.6 представлено сравнение результатов для разных параметров шумов и радиуса наблюдения.

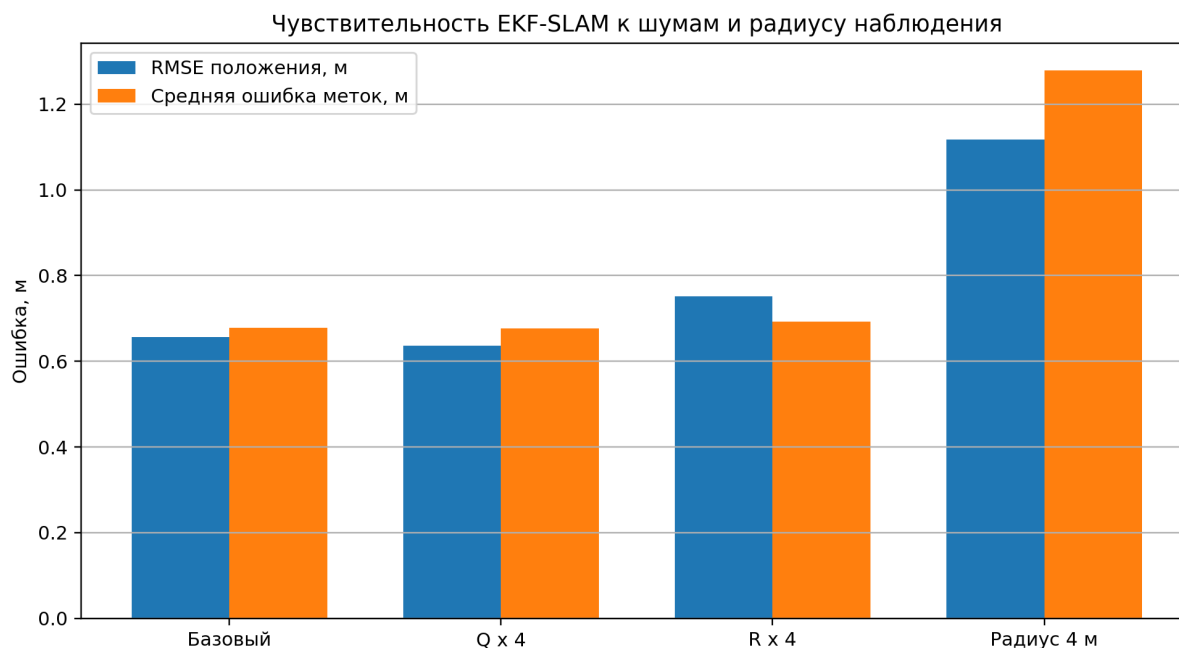


Рисунок 4.6. Сравнение RMSE положения и средней ошибки меток для разных экспериментов

Сводные результаты экспериментов приведены в табл. 4.2.

Таблица 4.2

Сравнение качества EKF-SLAM при различных параметрах

Эксперимент	$RMSE_{pos}$, м	$RMSE_{\theta}$, град	Средняя ошибка меток, м	Инициализировано меток	Среднее число видимых меток	Средний NIS
Базовый	0.6569	3.6529	0.6777	8	4.939	2.044
Увеличенный Q	0.6371	3.4102	0.6765	8	4.939	2.035
Увеличенный R	0.7513	3.9374	0.6927	8	4.757	1.992
Радиус 4 м	1.1179	11.2639	1.2788	8	2.341	2.055

Из табл. 4.2 видно, что в базовом эксперименте средняя ошибка положения робота составила 0.6569 м, а средняя ошибка координат меток составила 0.6777 м. Все восемь меток были успешно инициализированы.

При увеличении шума измерений Q значение $RMSE_{pos}$ составило 0.6371 м, а средняя ошибка меток составила 0.6765 м. В данном численном эксперименте ухудшения результата не произошло, так как траектория, радиус видимости и набор наблюдений остались достаточно информативными для коррекции состояния.

При увеличении шума процесса R ошибка положения увеличилась до 0.7513 м. Это связано с тем, что менее точная модель движения ухудшает качество априорного прогноза состояния робота между измерениями.

Наиболее заметное ухудшение наблюдается при уменьшении радиуса видимости до 4 м. В этом случае среднее число видимых меток снизилось с 4.939 до 2.341, а ошибка положения выросла до 1.1179 м. Средняя ошибка меток также увеличилась до 1.2788 м. Следовательно, для EKF-SLAM важны не только уровни шумов, но и достаточное количество наблюдаемых ориентиров.

5. Выводы

В ходе лабораторной работы был реализован алгоритм одновременной локализации и картирования на основе расширенного фильтра Калмана. Робот двигался в двумерной среде с восемью статическими метками и получал измерения дальности и азимута до видимых ориентиров.

В работе был реализован полный вариант EKF-SLAM, при котором координаты меток заранее неизвестны. При первом наблюдении метки ее координаты переводились из локальной системы координат робота в мировую систему координат, после чего метка добавлялась в общий вектор состояния фильтра.

В базовом эксперименте были инициализированы все восемь меток. Среднеквадратическая ошибка положения робота составила

$$RMSE_{pos} = 0.6569 \text{ м}$$

а средняя ошибка координат меток составила

$$\bar{e}_m = 0.6777 \text{ м}$$

Реализованный фильтр показал устойчивую работу: ошибка положения после начального переходного участка не возрастала неограниченно, а накопленное значение RMSE стабилизировалось. Это говорит о сходимости оценки при достаточном числе наблюдений меток.

Среднее значение критерия NIS для базового эксперимента равно

$$\overline{NIS} = 2.0445$$

Такой порядок величины является адекватным для двумерного измерения, состоящего из дальности и азимута. Следовательно, заданная ковариация измерений в базовом эксперименте в целом соответствует фактическим невязкам.

Проведенное исследование чувствительности показало, что на качество EKF-SLAM существенно влияет количество наблюдаемых меток. При уменьшении радиуса видимости до 4 м среднее число видимых ориентиров уменьшилось, а ошибка положения робота увеличилась до 1.1179 м. Следовательно, для устойчивой работы EKF-SLAM необходимы достаточно частые и информативные наблюдения ориентиров.

Ограничениями реализованного метода являются зависимость от начальной неопределенности, уровня шумов, радиуса видимости и корректности сопоставления измерений с уже известными метками. В данной работе идентификаторы меток считались известными, поэтому задача ассоциации данных отдельно не рассматривалась.

6. Исходный код

Listing 1: Код

```
1 import os
2 import numpy as np
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6 from matplotlib.patches import Ellipse
7
8
9 SEED = 42
10 DT = 0.1
11 N_STEPS = 700
12 MAX_RANGE = 6.5
13 BASE_DIR = os.path.dirname(os.path.abspath(__file__)) if "__file__" in globals()
14     else os.getcwd()
15 IMG_DIR = os.path.join(BASE_DIR, "img")
16
17 LANDMARKS_TRUE = np.array([
18     [3.5, 0.5],
19     [6.0, 3.5],
20     [3.2, 7.0],
21     [-1.0, 7.7],
22     [-4.2, 5.5],
23     [-4.4, 1.5],
24     [-1.0, -1.0],
25     [2.0, 3.8],
26 ], dtype=float)
27
28 Q_BASE = np.diag([0.12 ** 2, np.deg2rad(1.5) ** 2])
29 R_BASE = np.diag([0.02 ** 2, 0.02 ** 2, np.deg2rad(0.7) ** 2])
30 P0_ROBOT = np.diag([0.35 ** 2, 0.35 ** 2, np.deg2rad(8.0) ** 2])
31
32 def normalize_angle(a):
33     return (a + np.pi) % (2.0 * np.pi) - np.pi
34
35
36 def control_input(k):
37     t = k * DT
38     v = 0.75 + 0.08 * np.sin(0.20 * t)
```

```

39     omega = 0.19 + 0.04 * np.sin(0.17 * t)
40     return np.array([v, omega], dtype=float)
41
42
43 def motion_model(x, u, dt):
44     y = x.copy()
45     v, omega = u
46     theta = x[2]
47     y[0] = x[0] + v * dt * np.cos(theta)
48     y[1] = x[1] + v * dt * np.sin(theta)
49     y[2] = normalize_angle(x[2] + omega * dt)
50     return y
51
52
53 def predict(mu, P, u, R, dt):
54     n = mu.size
55     v, omega = u
56     theta = mu[2]
57
58     F = np.eye(n)
59     F[0, 2] = -v * dt * np.sin(theta)
60     F[1, 2] = v * dt * np.cos(theta)
61
62     mu = mu.copy()
63     mu[0] = mu[0] + v * dt * np.cos(theta)
64     mu[1] = mu[1] + v * dt * np.sin(theta)
65     mu[2] = normalize_angle(mu[2] + omega * dt)
66
67     P = F @ P @ F.T
68     P[:3, :3] += R
69     P = 0.5 * (P + P.T)
70     return mu, P
71
72
73 def measurement_model(robot_pose, landmark_pos):
74     x, y, theta = robot_pose
75     dx = landmark_pos[0] - x
76     dy = landmark_pos[1] - y
77     r = np.sqrt(dx * dx + dy * dy)
78     b = normalize_angle(np.arctan2(dy, dx) - theta)
79     return np.array([r, b], dtype=float)
80
81
82 def generate_measurements(x_true, landmarks, Q, max_range, rng):
83     measurements = []
84     sigma_r = np.sqrt(Q[0, 0])
85     sigma_b = np.sqrt(Q[1, 1])
86
87     for lm_id, lm in enumerate(landmarks):
88         z_true = measurement_model(x_true, lm)
89         if z_true[0] <= max_range:
90             z = z_true.copy()
91             z[0] += rng.normal(0.0, sigma_r)
92             z[1] = normalize_angle(z[1] + rng.normal(0.0, sigma_b))
93             measurements.append((lm_id, z))
94
95     return measurements
96
97
98 def initialize_landmark(mu, P, lm_id, z, Q, landmark_to_index):

```

```

99     r, b = z
100     x, y, theta = mu[:3]
101     alpha = normalize_angle(theta + b)
102
103     lm = np.array([
104         x + r * np.cos(alpha),
105         y + r * np.sin(alpha),
106     ], dtype=float)
107
108     n_old = mu.size
109     n_new = n_old + 2
110     mu_new = np.zeros(n_new)
111     mu_new[:n_old] = mu
112     mu_new[n_old:n_new] = lm
113
114     Gx = np.array([
115         [1.0, 0.0, -r * np.sin(alpha)],
116         [0.0, 1.0, r * np.cos(alpha)],
117     ])
118
119     Gz = np.array([
120         [np.cos(alpha), -r * np.sin(alpha)],
121         [np.sin(alpha), r * np.cos(alpha)],
122     ])
123
124     P_new = np.zeros((n_new, n_new))
125     P_new[:n_old, :n_old] = P
126
127     cross = Gx @ P[:3, :]
128     P_new[n_old:n_new, :n_old] = cross
129     P_new[:n_old, n_old:n_new] = cross.T
130     P_new[n_old:n_new, n_old:n_new] = Gx @ P[:3, :3] @ Gx.T + Gz @ Q @ Gz.T
131     P_new = 0.5 * (P_new + P_new.T)
132
133     landmark_to_index[lm_id] = n_old
134     return mu_new, P_new
135
136
137 def update_landmark(mu, P, lm_id, z, Q, landmark_to_index):
138     lm_index = landmark_to_index[lm_id]
139     x, y, theta = mu[:3]
140     mx, my = mu[lm_index:lm_index + 2]
141
142     dx = mx - x
143     dy = my - y
144     q = dx * dx + dy * dy
145     q = max(q, 1e-12)
146     sqrt_q = np.sqrt(q)
147
148     z_hat = np.array([
149         sqrt_q,
150         normalize_angle(np.arctan2(dy, dx) - theta),
151     ])
152
153     innovation = np.array([
154         z[0] - z_hat[0],
155         normalize_angle(z[1] - z_hat[1]),
156     ])
157
158     n = mu.size

```



```

159     H = np.zeros((2, n))
160
161     H[0, 0] = -dx / sqrt_q
162     H[0, 1] = -dy / sqrt_q
163     H[0, 2] = 0.0
164     H[1, 0] = dy / q
165     H[1, 1] = -dx / q
166     H[1, 2] = -1.0
167
168     H[0, lm_index] = dx / sqrt_q
169     H[0, lm_index + 1] = dy / sqrt_q
170     H[1, lm_index] = -dy / q
171     H[1, lm_index + 1] = dx / q
172
173     S = H @ P @ H.T + Q
174     K = P @ H.T @ np.linalg.inv(S)
175
176     mu = mu + K @ innovation
177     mu[2] = normalize_angle(mu[2])
178
179     I = np.eye(n)
180     P = (I - K @ H) @ P @ (I - K @ H).T + K @ Q @ K.T
181     P = 0.5 * (P + P.T)
182
183     nis = float(innovation.T @ np.linalg.inv(S) @ innovation)
184     return mu, P, nis
185
186
187 def covariance_ellipse_params(P2, n_std=2.0):
188     vals, vecs = np.linalg.eigh(P2)
189     vals = np.maximum(vals, 0.0)
190     order = vals.argsort()[::-1]
191     vals = vals[order]
192     vecs = vecs[:, order]
193     angle = np.degrees(np.arctan2(vecs[1, 0], vecs[0, 0]))
194     width = 2.0 * n_std * np.sqrt(vals[0])
195     height = 2.0 * n_std * np.sqrt(vals[1])
196     return width, height, angle
197
198
199 def run_ekf_slam(q_factor=1.0, r_factor=1.0, max_range=MAX_RANGE, seed=SEED):
200     rng = np.random.default_rng(seed)
201     Q = Q_BASE * q_factor
202     R = R_BASE * r_factor
203
204     x_true = np.array([0.0, 0.0, np.deg2rad(8.0)], dtype=float)
205     mu = np.array([0.25, -0.25, np.deg2rad(4.0)], dtype=float)
206     P = P0_ROBOT.copy()
207     landmark_to_index = {}
208
209     true_history = np.zeros((N_STEPS + 1, 3))
210     est_history = np.zeros((N_STEPS + 1, 3))
211     visible_count_history = np.zeros(N_STEPS + 1)
212     nis_history = []
213
214     true_history[0] = x_true
215     est_history[0] = mu[:3]
216
217     for k in range(1, N_STEPS + 1):
218         u = control_input(k - 1)

```

```

219     process_noise = rng.multivariate_normal(np.zeros(3), R)
220     x_true = motion_model(x_true, u, DT)
221     x_true += process_noise
222     x_true[2] = normalize_angle(x_true[2])
223
224
225     mu, P = predict(mu, P, u, R, DT)
226
227     measurements = generate_measurements(x_true, LANDMARKS_TRUE, Q,
228                                         max_range, rng)
229     visible_count_history[k] = len(measurements)
230
231     for lm_id, z in measurements:
232         if lm_id not in landmark_to_index:
233             mu, P = initialize_landmark(mu, P, lm_id, z, Q,
234                                         landmark_to_index)
235         else:
236             mu, P, nis = update_landmark(mu, P, lm_id, z, Q,
237                                         landmark_to_index)
238             nis_history.append(nis)
239
240     true_history[k] = x_true
241     est_history[k] = mu[:3]
242
243     pos_errors = est_history[:, :2] - true_history[:, :2]
244     theta_errors = np.array([normalize_angle(a) for a in est_history[:, 2] -
245                             true_history[:, 2]])
246     pos_error_norm = np.linalg.norm(pos_errors, axis=1)
247     rmse_position = float(np.sqrt(np.mean(pos_error_norm ** 2)))
248     rmse_x = float(np.sqrt(np.mean(pos_errors[:, 0] ** 2)))
249     rmse_y = float(np.sqrt(np.mean(pos_errors[:, 1] ** 2)))
250     rmse_theta_deg = float(np.rad2deg(np.sqrt(np.mean(theta_errors ** 2))))
251
252     landmark_errors = []
253     estimated_landmarks = {}
254     for lm_id, start_index in landmark_to_index.items():
255         est_lm = mu[start_index:start_index + 2]
256         estimated_landmarks[lm_id] = est_lm
257         landmark_errors.append(np.linalg.norm(est_lm - LANDMARKS_TRUE[lm_id]))
258
259     mean_landmark_error = float(np.mean(landmark_errors)) if landmark_errors
260     else np.nan
261     max_landmark_error = float(np.max(landmark_errors)) if landmark_errors else
262     np.nan
263
264     result = {
265         "true_history": true_history,
266         "est_history": est_history,
267         "pos_errors": pos_errors,
268         "theta_errors": theta_errors,
269         "pos_error_norm": pos_error_norm,
270         "visible_count_history": visible_count_history,
271         "nis_history": np.array(nis_history),
272         "mu": mu,
273         "P": P,
274         "landmark_to_index": landmark_to_index,
275         "estimated_landmarks": estimated_landmarks,
276         "metrics": {
277             "q_factor": q_factor,
278             "r_factor": r_factor,

```

```

273         "max_range": max_range,
274         "steps": N_STEPS,
275         "landmarks_total": LANDMARKS_TRUE.shape[0],
276         "landmarks_initialized": len(landmark_to_index),
277         "rmse_position": rmse_position,
278         "rmse_x": rmse_x,
279         "rmse_y": rmse_y,
280         "rmse_theta_deg": rmse_theta_deg,
281         "mean_landmark_error": mean_landmark_error,
282         "max_landmark_error": max_landmark_error,
283         "mean_visible_landmarks": float(np.mean(visible_count_history[1:])),
284         "mean_nis": float(np.mean(nis_history)) if nis_history else np.nan,
285     },
286 }
287 return result
288
289
290 def plot_nominal_result(result, img_dir=IMG_DIR):
291     os.makedirs(img_dir, exist_ok=True)
292
293     true_history = result["true_history"]
294     est_history = result["est_history"]
295     P = result["P"]
296     landmark_to_index = result["landmark_to_index"]
297     estimated_landmarks = result["estimated_landmarks"]
298     pos_errors = result["pos_errors"]
299     theta_errors = result["theta_errors"]
300     pos_error_norm = result["pos_error_norm"]
301     visible_count_history = result["visible_count_history"]
302     nis_history = result["nis_history"]
303     t = np.arange(N_STEPS + 1) * DT
304
305     fig, ax = plt.subplots(figsize=(9, 7))
306     ax.plot(true_history[:, 0], true_history[:, 1], label="Истинная траектория")
307     ax.plot(est_history[:, 0], est_history[:, 1], "--", label="Оцененная траектория EKF-SLAM")
308     ax.scatter(LANDMARKS_TRUE[:, 0], LANDMARKS_TRUE[:, 1], marker="x", s=140, label="Истинные метки")
309
310     if estimated_landmarks:
311         ids = sorted(estimated_landmarks.keys())
312         est_lm_arr = np.array([estimated_landmarks[i] for i in ids])
313         ax.scatter(est_lm_arr[:, 0], est_lm_arr[:, 1], marker="x", s=70, label="Оцененные метки")
314
315         for lm_id in ids:
316             idx = landmark_to_index[lm_id]
317             lm = estimated_landmarks[lm_id]
318             width, height, angle = covariance_ellipse_params(P[idx:idx + 2, idx:idx + 2], n_std=2.0)
319             ell = Ellipse(xy=lm, width=width, height=height, angle=angle, fill=False, linewidth=1.2)
320             ax.add_patch(ell)
321             ax.text(lm[0] + 0.12, lm[1] + 0.12, str(lm_id), fontsize=9)
322
323     ax.set_title("EKF-SLAM: траектория работы и картаметок ")
324     ax.set_xlabel("x, м")
325     ax.set_ylabel("y, м")
326     ax.axis("equal")
327     ax.grid(True)

```

```

328 ax.legend()
329 fig.tight_layout()
330 fig.savefig(os.path.join(img_dir, "trajectory_landmarks.png"), dpi=220)
331 plt.close(fig)
332
333 fig, ax = plt.subplots(figsize=(9, 5))
334 ax.plot(t, pos_errors[:, 0], label="Ошибка x")
335 ax.plot(t, pos_errors[:, 1], label="Ошибка y")
336 ax.plot(t, np.rad2deg(theta_errors), label="Ошибка theta, град")
337 ax.set_title("Ошибки оценки состояния робота ")
338 ax.set_xlabel("t, с")
339 ax.set_ylabel("Ошибка")
340 ax.grid(True)
341 ax.legend()
342 fig.tight_layout()
343 fig.savefig(os.path.join(img_dir, "robot_state_errors.png"), dpi=220)
344 plt.close(fig)
345
346 cumulative_rmse = np.sqrt(np.cumsum(pos_error_norm ** 2) / np.arange(1,
    N_STEPS + 2))
347 fig, ax = plt.subplots(figsize=(9, 5))
348 ax.plot(t, pos_error_norm, label="Текущая ошибка положения ")
349 ax.plot(t, cumulative_rmse, "--", label="Накопленный RMSE")
350 ax.set_title("Ошибка положения робота и RMSE")
351 ax.set_xlabel("t, с")
352 ax.set_ylabel("Ошибка, м")
353 ax.grid(True)
354 ax.legend()
355 fig.tight_layout()
356 fig.savefig(os.path.join(img_dir, "position_rmse.png"), dpi=220)
357 plt.close(fig)
358
359 fig, ax = plt.subplots(figsize=(9, 4))
360 ax.plot(t, visible_count_history)
361 ax.set_title("Количество видимых меток во времени ")
362 ax.set_xlabel("t, с")
363 ax.set_ylabel("Количество меток")
364 ax.grid(True)
365 fig.tight_layout()
366 fig.savefig(os.path.join(img_dir, "visible_landmarks.png"), dpi=220)
367 plt.close(fig)
368
369 if nis_history.size > 0:
370     fig, ax = plt.subplots(figsize=(9, 4))
371     ax.plot(nis_history)
372     ax.axhline(2.0, linestyle="--", label="Ожидаемое среднее NIS для 2
        измерений")
373     ax.set_title("NIS для измерений дальности азимут -")
374     ax.set_xlabel("Номер обновления")
375     ax.set_ylabel("NIS")
376     ax.grid(True)
377     ax.legend()
378     fig.tight_layout()
379     fig.savefig(os.path.join(img_dir, "nis.png"), dpi=220)
380     plt.close(fig)
381
382
383 def run_sensitivity_experiments(img_dir=IMG_DIR):
384     os.makedirs(img_dir, exist_ok=True)
385     experiments = [

```

```

386         ("Базовый", 1.0, 1.0, MAX_RANGE),
387         ("Q x 4", 4.0, 1.0, MAX_RANGE),
388         ("R x 4", 1.0, 4.0, MAX_RANGE),
389         ("Радиус 4 м", 1.0, 1.0, 4.0),
390     ]
391
392     rows = []
393     for name, q_factor, r_factor, max_range in experiments:
394         res = run_ekf_slam(q_factor=q_factor, r_factor=r_factor, max_range=
395             max_range, seed=SEED)
396         m = res["metrics"]
397         rows.append([
398             name,
399             m["rmse_position"],
400             m["rmse_theta_deg"],
401             m["mean_landmark_error"],
402             m["landmarks_initialized"],
403             m["mean_visible_landmarks"],
404             m["mean_nis"],
405         ])
406
407     labels = [r[0] for r in rows]
408     rmse_values = [r[1] for r in rows]
409     landmark_values = [r[3] for r in rows]
410
411     x = np.arange(len(labels))
412     width = 0.36
413     fig, ax = plt.subplots(figsize=(9, 5))
414     ax.bar(x - width / 2, rmse_values, width, label="RMSE положения, м")
415     ax.bar(x + width / 2, landmark_values, width, label="Средняя ошибка меток, м")
416     ax.set_title("Чувствительность EKF-SLAM к шумам радиуса наблюдения ")
417     ax.set_ylabel("Ошибка, м")
418     ax.set_xticks(x)
419     ax.set_xticklabels(labels)
420     ax.grid(True, axis="y")
421     ax.legend()
422     fig.tight_layout()
423     fig.savefig(os.path.join(img_dir, "noise_sensitivity.png"), dpi=220)
424     plt.close(fig)
425
426     table_path = os.path.join(img_dir, "sensitivity_results.csv")
427     with open(table_path, "w", encoding="utf-8") as f:
428         f.write("experiment,rmse_position_m,rmse_theta_deg,mean_landmark_error_m,
429             initialized_landmarks,mean_visible_landmarks,mean_nis\n")
430         for row in rows:
431             f.write(",".join([str(row[0])] + [f"{v:.6f}" if isinstance(v, float)
432                 else str(v) for v in row[1:]]) + "\n")
433
434     return rows
435
436 def print_metrics(metrics):
437     print("\nИтоговые метрики базового эксперимента ")
438     print(f"Количество шагов: {metrics['steps']}")
439     print(f"Всего меток: {metrics['landmarks_total']}")
440     print(f"Инициализировано меток: {metrics['landmarks_initialized']}")
441     print(f"Среднее число видимых меток : {metrics['mean_visible_landmarks']:.3f}")
442     print(f"RMSE положения робота : {metrics['rmse_position']:.4f} м")

```

```

441     print(f"RMSE по x: {metrics['rmse_x']:.4f} м")
442     print(f"RMSE по y: {metrics['rmse_y']:.4f} м")
443     print(f"RMSE по theta: {metrics['rmse_theta_deg']:.4f} град")
444     print(f"Средняя ошибка по меткам : {metrics['mean_landmark_error']:.4f} м")
445     print(f"Максимальная ошибка по меткам : {metrics['max_landmark_error']:.4f} м")
446     print(f"Среднее NIS: {metrics['mean_nis']:.4f}")
447
448
449 def print_sensitivity_table(rows):
450     print("\Исследования чувствительности")
451     print(f"Эксперимент{'':<14} {'RMSE, м':>10} {'theta, град':>12} Ошибка{' 'меток, м':>16} Метки{'':>8} Видно{'':>8} {'NIS':>8}")
452
453     for row in rows:
454         name, rmse, theta_rmse, lm_err, initialized, visible, nis = row
455         print(f"{name:<14} {rmse:>10.4f} {theta_rmse:>12.4f} {lm_err:>16.4f} {initialized:>8} {visible:>8.3f} {nis:>8.3f}")
456
457
458 def main():
459     os.makedirs(IMG_DIR, exist_ok=True)
460
461     nominal = run_ekf_slam(q_factor=1.0, r_factor=1.0, max_range=MAX_RANGE, seed=SEED)
462     plot_nominal_result(nominal, IMG_DIR)
463     rows = run_sensitivity_experiments(IMG_DIR)
464
465     print_metrics(nominal["metrics"])
466     print_sensitivity_table(rows)
467     print("\Сохраненные файлы:")
468     print(f" {IMG_DIR}/trajectory_landmarks.png")
469     print(f" {IMG_DIR}/robot_state_errors.png")
470     print(f" {IMG_DIR}/position_rmse.png")
471     print(f" {IMG_DIR}/visible_landmarks.png")
472     print(f" {IMG_DIR}/nis.png")
473     print(f" {IMG_DIR}/noise_sensitivity.png")
474     print(f" {IMG_DIR}/sensitivity_results.csv")
475
476
477 if __name__ == "__main__":
478     main()

```